금융 시스템 아키텍처의 과거 현재 그리고 미래

Produced by Tae Young Lee

금융 시스템

금융 시스템

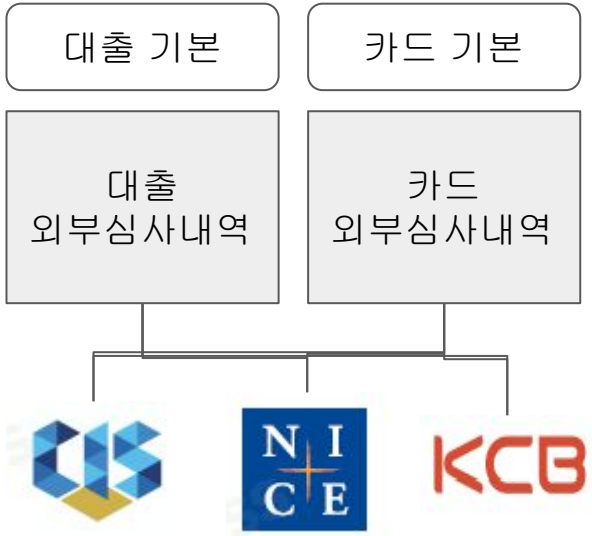
금융은 기본적으로 규제 산업이다.

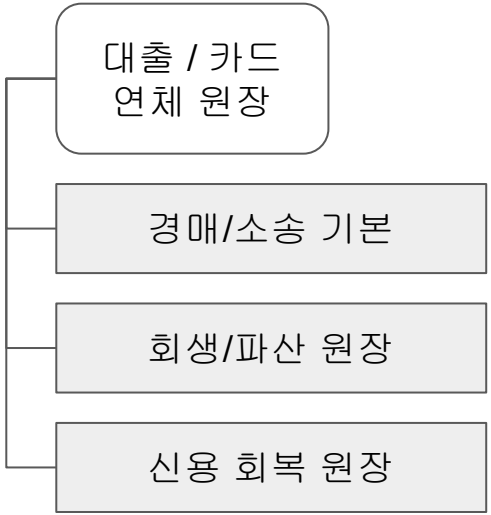
금융 업무는 업무별 기준 준칙이 있고, 제약사항과 법률적 기준을 준수해야 한다.

그렇기 때문에 DB Table을 유기적으로 엮어 놓으면 관리가 용의하지 않고, 규제나 법률 사항 변경 시 즉시 적용이 불가하다.

그래서 업무의 성격이나 프로세스에 따라 테이블의 통합과 분리가 고려되어야 하고 업무시스템 장애 방지를 위해서 느슨한 연결을 해 놓거나, 단일 DB로 아키텍처를 수립할 수 밖에 없다.

테이블의 분리와 통합





심사는 평가항목이 대출/카드가 상이한 항목이 많다
테이블을 심사내역 하나로 통합했을 때 불리한 점이 많다.

연체 이후는 대출이나 카드나 연체금액/연체일 등 공통적인 항목이 많다.
또 이후에 발생하는 법적절차 테이블들은 고객별 Data이다.

금융에서 현재 잘 관리되고 있는 테이블 목록

상품	금리	수수료
여신	여신	여신
수신	수신	수신
카드	카드	카드

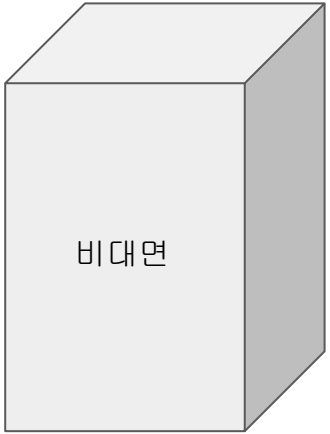
상품, 금리, 수수료가 각 팩토리에서 통합되어 관리됨

단일성 코드	목록성 코드	
Meta System	공통코드	여신코드
	회계코드	수신코드
		카드코드

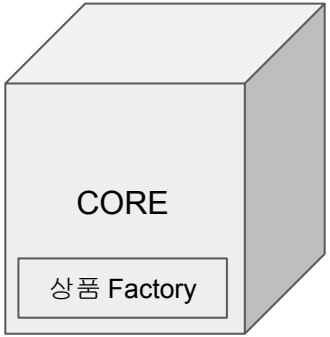
고객 정보는 고객에 통합되어 관리되고 있어, 개인정보가 효율적으로 관리되고 있다.

위와 같이 데이터 관리의 정답은 되도록 공통분모를 모아서 테이블을 설계하자.

데이터량/Transaction의 유형을 고려하여 공통분모를 모으자

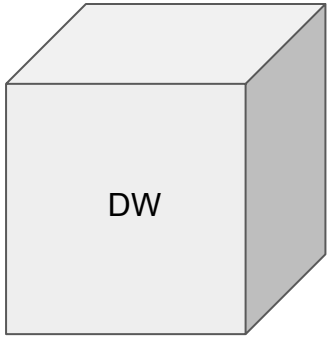


전자금융서버

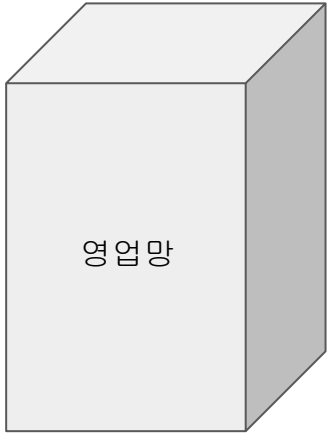


통합단말서버

은행	외화	골드	신탁	종금
외환법 (수수료 / 금리 / 상품)			신탁법	종금법



Data Warehouse



단위시스템

원천 Oracle: ARCHIVELOG + Supplemental Logging 활성화

Full Load로 DW 초기 적재 (Data Pump/DBLink)

CDC: GoldenGate 또는 LogMiner 기반 변경 반영

DW 내 SCD 반영 및 품질 검증/모니터링

Stage → Integration → Mart 구조로 분석 활용

CDC (Change Data Capture): 원천 DB에서 발생한 변경(INSERT/UPDATE/DELETE)을 실시간·근실시간·또는 배치로 식별·추출하는 기술/패턴입니다.

SCD (Slowly Changing Dimension): DW(데이터웨어하우스) 내에서 차원 테이블(고객, 상품 등)의 변경 이력을 어떻게 관리할지에 대한 설계 패턴입니다.

CDC의 주요 방식

1. **로그 기반(Log-based CDC)** — 권장 (예: Oracle GoldenGate, Debezium, LogMiner)
   - DB의 redo/transaction log(Oracle: redo/archivelog)를 읽어 변경 이벤트를 추출.
   - 장점: 낮은 원본 부하, 트랜잭션 경계 유지, 고신뢰성.
   - 단점: 초기 설정(예: supplemental logging) 필요, 로그 포맷 의존.
2. **트리거 기반(Trigger-based CDC)**
   - 테이블의 INSERT/UPDATE/DELETE 트리거를 만들어 변경을 별도 테이블/audit queue에 기록.
   - 장점: 단순 구현(소규모), 세밀한 제어 가능.
   - 단점: 원본 DB 부하 증가, 트랜잭션 지연 가능, 관리 어려움(다수 테이블).
3. **타임스탬프 기반(Polling) / 쿼리 기반**
   - 변경일자 컬럼(last_update_ts) 또는 증분 키를 주기적으로 쿼리해 변경 추출.
   - 장점: 구현 단순, DB 로그 접근 불필요.
   - 단점: 정확성/일관성 문제(동시성), 놓침 가능성, 지연 큼.
4. **Dump/Batch 기반 (Full Extract + diff)**
   - 정기적으로 전체 덤프 후 diff(또는 truncate+reload).
   - 장점: 단순, 데이터 일관성 확보 쉬움.
   - 단점: 비용·시간 많이 듦, 실시간 요구 불가.

개선 포인트

근본적인 개선을 하기 위해서는 도메인 전문가의 도움이 필요하다.

해당 데이터 셋이 파생되기 까지 어떤 절차와 규제를 통하여 산출되었는지를 알아야 한다.

그리고 가능하면 정제된 데이터가 아닌 원천 데이터 확보가 무엇보다 선행되어야 한다.

그리고 원천 데이터를 통해 정합성 체크 및 규제 내역 점검을 수행하고, 이를 통해 규제 내역에 대한 변경내역을 하나의 창구로 변경 시키고 유지되고 관리될 수 있도록 기존 시스템 아키텍처를 변경해 나가야 한다.

위 내역이 선행되어야만 여러가지 모델링 기법을 통해 성능 평가를 하기가 용의하다.

카카오 뱅크 아키텍처

Front-end	
Javascript Frameworks	Backbone.js
Javascript Template	Underscore.js
Javascript Libraries	jQuery.js, Moment.js, crypto-js, easyXDM, ES6-Promise, Web2App
Web Font	YoonGothic 720/750

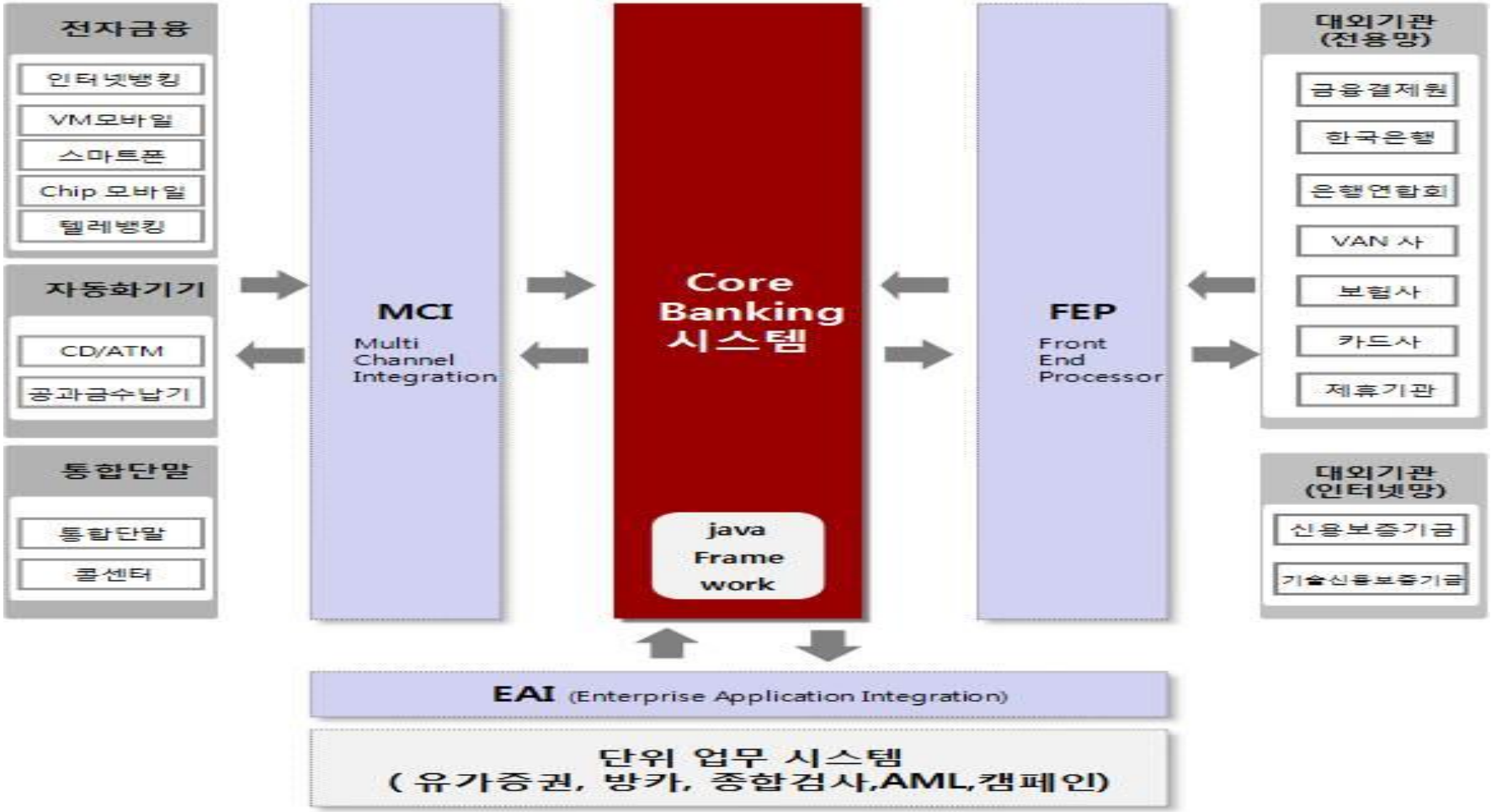
CDN and Security	
CDN	Akamai
Security	AhnLab Safe Transaction, HSM(hardware security modules), XecureKeyManager

Back-end	
Operating Systems	Linux
Web Server	Nginx
Databases	MySQL, Percona Server for MySQL, Oracle
Hadoop Ecosystem	Hadoop, HBase, Spark
Languages	Java, Swift, JavaScript

Utilities	
DB Tool	Tadpole DB Hub
VCS	Git

Core banking system	
전북은행	LG CNS가 개발한 국내 최초의 JAVA 기반 코어 뱅킹

참고링크 : https://blog.gaerae.com/2017/08/kakaobank-architecture.html



참고 링크 :http://m.bikorea.net/news/articleView.html?idxno=14530

구분	용도	도입 및 구축 요건
시스템 S/W	OS	❍ 상용리눅스 (예: RedHat Enterprise)
	DBMS	❍ 상용RDBMS Cluster : 계정계 DB, RDW, ADW
		❍ 상용RDBMS Single : 개발/테스트
		❍ 채널계 : MARIADB
		❍ PPAS : 상기 이외의 모든 시스템DB (불가할 경우 사유를 명기할 것)
	Web/WAS	❍ 상용 미들웨어
통합 및 I/F S/W	MCI	❍ 다양한 형태의 채널 (콜센터, 통합단말, 인터넷, 모바일) 지원
	CDC	❍ 계정=>RDW
	EAI	❍ 어플리케이션 통합 솔루션
	FEP	❍ 다양한 통신 프로토콜 지원
	ETL	❍ 당행이 적용 예정인DBMS 지원
	파일전송	❍ 암호화 전송
		❍ 스크립팅 지원
개발 S/W	프레임워크	❍ 개발/테스트/운영 환경 제공
	통합단말	❍ 사용자 환경에 최적화된 솔루션 제시
	BI/OLAP	❍ 리포팅, 향후 BI Portal 확장 구현
	리포팅툴	❍ 차트, 함수, 표편집, 쿼리 기능 등 지원
		❍ MS-오피스, 한컴 오피스, PDF 등 포맷 변환 기능 지원
	메타데이터관리	❍ 모델링 솔루션과 연계 기능 제공
	UML모델링 도구	❍ 개발용 임시 라이선스 포함
	데이터모델링 도구	❍ 메타 솔루션과 연계기능 제공
		❍ 개발용 임시 라이선스 포함
	영향도 분석	❍ 다양한 언어 및 DBMS 지원
관리 S/W	SMS/NMS	❍ 서버 및 네트워크 관리
	DB 관리	❍ DB관리 및 모니터링 솔루션
		❍ 당행이 적용 예정인 DBMS 지원
	Web/WAS 관리	❍ 당행이 적용 예정인 Web/WAS 지원
	형상관리	❍ 버전관리 및 배포 기능 지원
	백업관리	❍ 데이터 백업을 위한 라이선스
		❍ VT에 따른 백업 라이선스
	작업관리	❍ 배치 작업관리 솔루션
	ITSM	❍ 통합인프라 서비스 모니터링
		❍ 다양한 이종환경(다양한 DBMS)을 통합관리
		❍ 형상관리 연동
	...	
보안 S/W	SSO/EAM	❍ 사용자수를 고려하여 적정한 솔루션을 제시 할 것
	DB암호화	❍ 성능 최적화된 솔루션 제시
	전문암호화	❍ 다양한 환경 지원

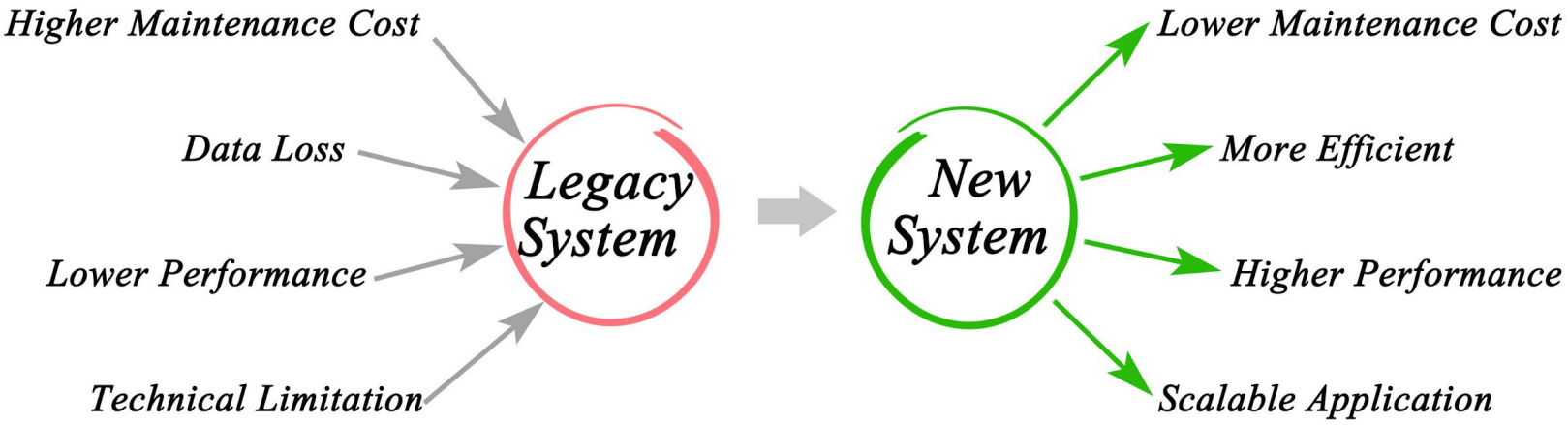
구분	1단계 구축 범위	비고
계정계	• 고객 • 상품 • 여신 • 수신 • 외환(제휴) : 해외송금,환전 • 체크카드(제휴) • 채권사후관리 • 자금 • 전자금융 • 계정공통/계리 : C/C, 계리, 코드, 문서/이미지 관리 등	상기 명시된 1단계 범위 : 개혁을 위해 필요한 시스템은 구축 필수 대상임
경영관리/정보분석	• EDW : RDW, ADW, 리포팅 용 통합마트 • 데이터 분석 : 대내외 리포팅	대외보고서 부문은 필수
	• 경영관리 : 인사, 급여, 예산 등	내부개발 예정
	• 재무회계	업무컨설팅 예정
	• 세무회계	
	• 지주사 보고	한투지주보고
	• 영업/마케팅 : CSS(신용평가시스템), CRM	별도 사업추진 예정
	• 리스크 : ALM, 신용, 시장, 운용	업무컨설팅 예정
	• 준법감시 : 종합감사, 상시감시, AML, FATCA, FDS(이상거래탐지시스템)	
	• 경영지원 : 내부포털(그룹웨어, 전자결재 등)	내부개발 예정
채널 및 대외	• 콜센터	
	• 대고객채널 : 인터넷 뱅킹, 모바일 뱅킹, 홈페이지	내부개발 예정
	• 대외계	
IT 인프라	• IT 인프라 구축 (H/W, S/W, N/W)	
	• 보안	별도 사업추진 예정
	• 전산실 구축	IDC 활용 예정

참고 링크 :http://m.bikorea.net/news/articleView.html?idxno=14530

금융사	증권사	은행	카드사
목적	투자	저축	소비
기간	단기/중기	수익 실현 / 목돈 마련 (장기 보유)	단기
상품 유형	투자 상품	구조화 상품	소비재
행태	가입		구매
	Buy / Sell		Buy / Not Buy

Infra as a model service

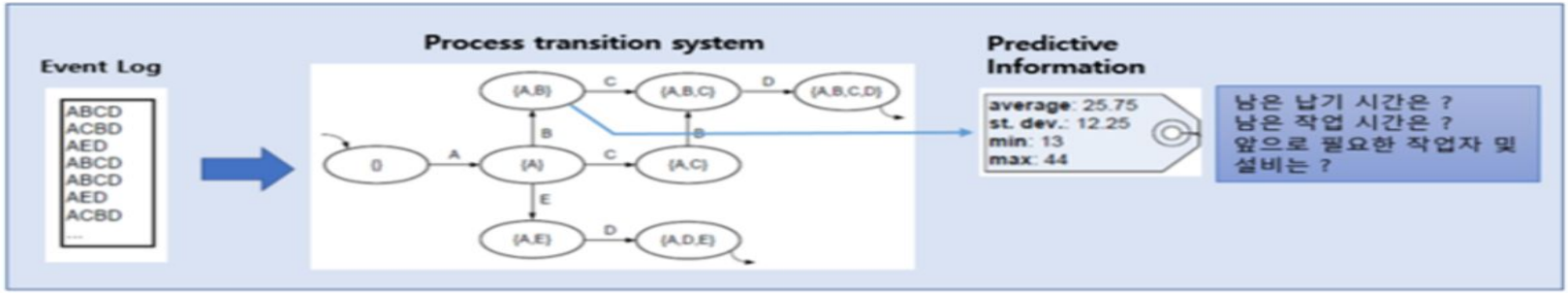
Infra Structure



출처 : https://www.cbxsoftware.com/is-retail-product-lifecycle-management-retail-plm-obsolete/legacy-system/

Legacy System

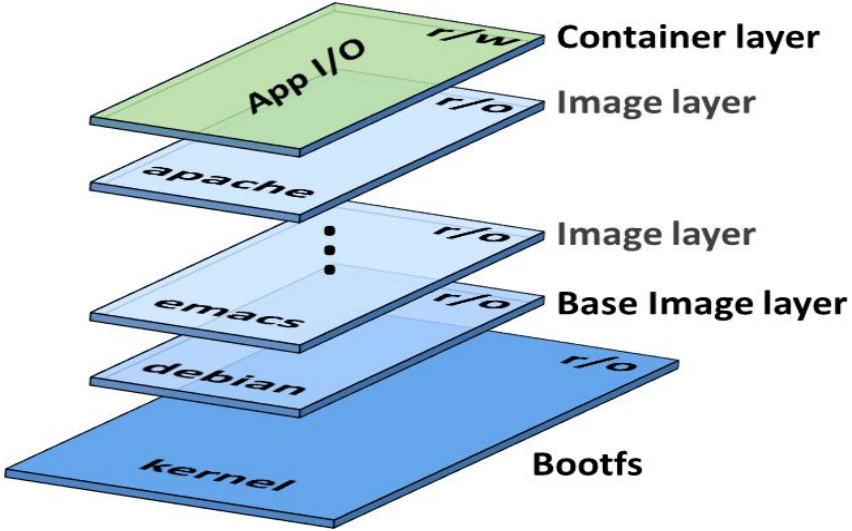
업무 시스템 중심의 데이터 프로세스



참고 : Time Prediction Based on Process Mining, W.M.P. van der Aalst, M.Song

New System

신기술 적용이 용의한 Data Transfrom및 모델 트레이닝이 가능한 인프라 환경



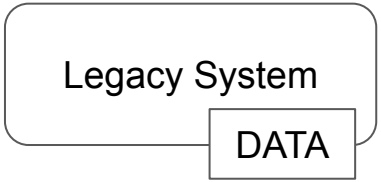
Kubernetes Ochestration

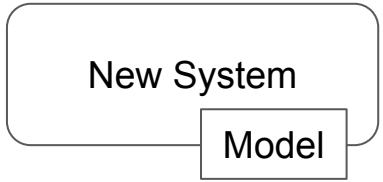
- POD Management
- Resource Control
- CPU/Memory Limits
- Eviction Management
- GPU Allocation

Dependency - Hardware 구성

이미지 출처 : http://neokobo.blogspot.com/2017/03/docker-container.html

Legacy System과 New System의 연결 고리





Dependency Injection
Input Data Generation
Output Data 전송

Data Cycle
- Delayed Batch
- Scheduling Batch

Model Cycle
- Real Time Inference
- Training Time 단축

WHY?
업무 시스템 안정성

WHY?
Model 운영 신뢰성 확보

제약적인 인프라 활용 시 유의해야 할 점

Model Size 관리

Task 별 Model Process 관리 (이를 통한 Inference Time 추정)

Legacy System에서 활용한 Data Size 확인 및 네트워크 Traffic 추정 필요
(New System과의 원활한 Interface방법 확보)

Model Training시 GPU 자원 부족 현상으로 인한 대응 방안 강구
(기존 학습 weights load되어 재학습 가능하도록 구성,
Epoch number restoring되도록 코드 구현)

Model Inference에 영향을 미치는 Infra요소

GPU (가용한 GPU확인 및 Model과 결합하는 Base Image Componet 효율적 구성)

Network (Network 지연 요소 확인)

Model Size

Memory Limitation : 모델이 커지면서 가장 먼저 문제가 될 수 있는 것은 메모리 사이즈입니다. BERT 파생 모델들의 사이즈가 점점 커지면서 하나의 GPU에서 큰 모델을 학습하는 것이 점점 어려워지고 있습니다 [7]. 또한 자연어 처리 분야에서 큰 배치 사이즈가 학습에 효과적이라는 의견이 나오면서 [8] 사전 학습에 사용되는 배치 크기가 점점 커지는 추세를 보이고 있고[9,10], 이러한 배치 사이즈의 증가는 메모리에 많은 부담이 되고 있습니다.

Training/Inference Speed : 학습에 필요한 gradient는 모델의 크기에 비례하기 때문에 분산 학습을 통해 학습 속도를 올리더라도, 모델이 커짐에 따라 학습에 보다 많은 시간이 소요되게 됩니다. 학습은 한 번만 진행되므로 시간이 오래 걸려도 괜찮을 수 있지만, 모델 크기가 증가하면서 추론에 걸리는 시간 역시 늘어나기 때문에 문제가 될 수 있습니다.

Worse Performance : 이런 문제점을 해결하기 위한 한 가지 접근 방법 중 하나는 분산 학습입니다. 기존의 많은 연구들은 1)데이터 병렬화와 2)모델 병렬화 방식과 같이 복수의 GPU를 사용한 학습을 통해 이 문제를 해결하려 하였습니다 하지만 분산 학습으로 모델을 학습하더라도 여전히 문제는 남습니다. 같은 데이터에서 단순히 모델만을 키운다고 성능이 계속해서 증가하지는 않기 때문입니다. [11]. 지나치게 큰 모델은 과적합(overfitting)하기 쉽고 [12], 이를 막기 위해서는 더 많은 데이터를 사용하거나, 정규화(regularization) 방법을 도입하여 해결할 필요가 있습니다 [13].

Practical problems : 분산 학습을 한다고 하더라도 모델이 커짐에 따라, 많은 GPU를 준비해야 하는 것은 작은 회사/연구소/대학원 등에서는 부담이 될 수 있습니다. 또한, 모바일/자동차 같은 환경에서는 휴대성, 전력 소모 등의 이유로 GPU 사용에 제한을 많이 받을 수 있어, 이와 같은 실용적인 측면에서의 문제도 고려해야 할 필요가 있습니다.

출처 : https://blog.est.ai/2020/03/딥러닝-모델-압축-방법론과-bert-압축/

모델 압축을 위한 기본적인 접근 방법들

가지치기 (Pruning) : 학습 후에 불필요한 부분을 제거하는 방식으로, 가중치의 크기에 기반한 제거, 어텐션 헤드 제거, 레이어 제거 등 여러가지 방법을 사용합니다. 몇몇 방법은 prunability를 제거하기 위해 학습 중에 정규화 방법을 도입하기도 합니다 (layer dropout).

가중치 분해 (Weight Factorization) : 가중치 행렬을 분해하여 두 개의 작은 행렬의 곱으로 근사하는 방법입니다. 이 방법은 행렬이 낮은 랭크를 가지도록 하는 제약조건(low-rank constraint)을 도입합니다. 가중치 분해는 토큰 임베딩이나 feed-forward / self-attention layer의 파라미터에 적용할 수 있습니다.

지식 증류 (Knowledge Distillation) : 미리 잘 학습된 큰 네트워크(Teacher network)로부터 실제로 사용하고자 하는 작은 네트워크(Student network)를 학습시키는 방식입니다. 훨씬 작은 Transformer 모델을 pre-training / downstream-data에 대해 기본부터 학습시킵니다. 원래대로라면 학습이 잘 안 되지만, fully-sized model의 값을 soft label로 사용하면 최적화가 더 잘 이루어집니다. 몇몇 방법들은 추론 시간을 빠르게 하기 위해 BERT를 다른 형태의 모델(LSTM, etc.)로 증류하기도 합니다. 이외 다른 방법들은 teacher에 대해 더 자세히 분석하여 출력값 뿐만 아니라, 가중치 행렬이나 hidden activation 값들을 사용하기도 합니다.

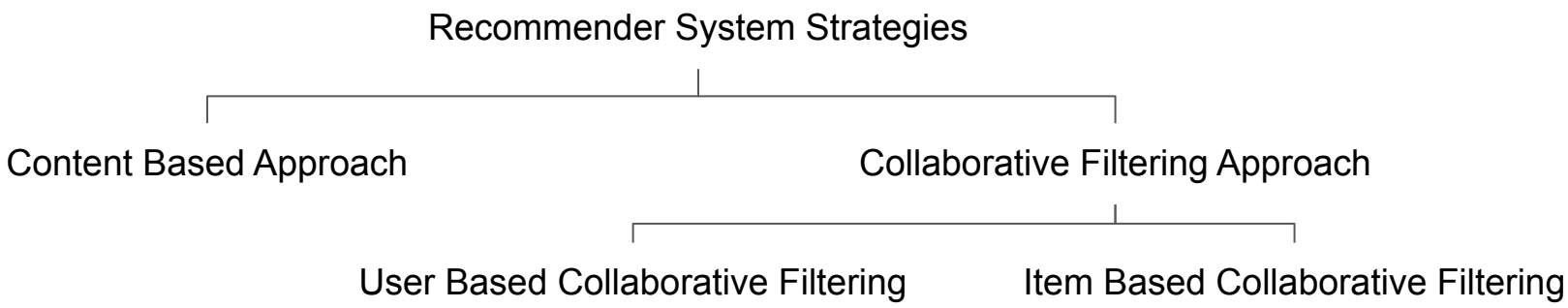
가중치 공유 (Weight Sharing) : 모델의 일부 가중치들을 다른 파라미터들과 공유하는 방식입니다. 예를 들어 ALBERT는 BERT의 self-attention layer와 같은 가중치 행렬들을 사용하고 있습니다. (ALBERT는 구글이 작년 9월 발표한 BERT의 새로운 버전으로, BERT보다 가볍고 처리 속도가 높아진 점이 특징입니다.)

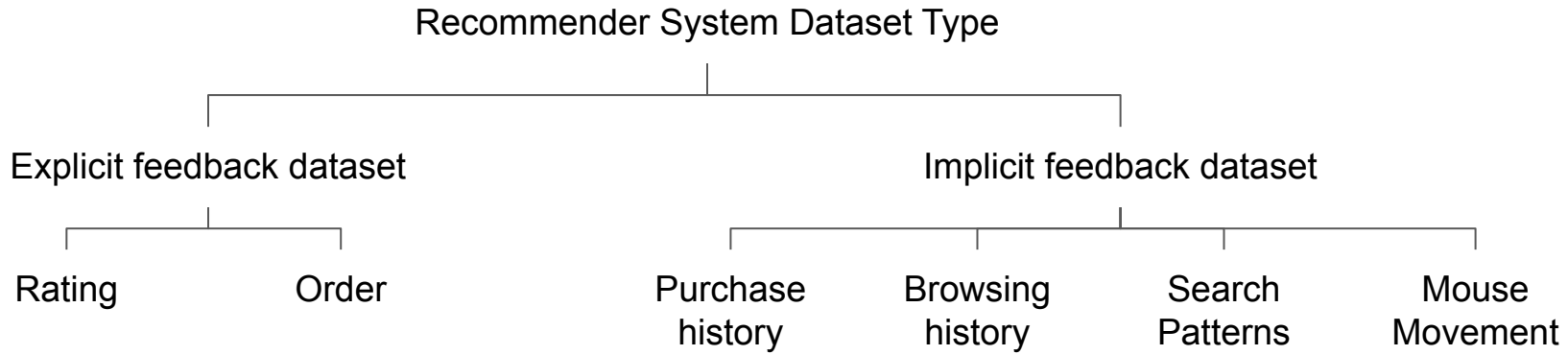
양자화 (Quantization) : 부동 소수점 값을 잘라내서 더 적은 비트만을 사용하는 방식입니다. 양자화된 값은 학습 과정 중에 배울 수도 있고, 학습 후에 양자화될 수도 있습니다.

Pre-train vs. Downstream : 일부 방법들은 BERT를 특정 downstream task에만 맞게 압축하지만, BERT를 task와 무관하게 압축하는 방법들도 있습니다.

출처 : https://blog.est.ai/2020/03/딥러닝-모델-압축-방법론과-bert-압축/

일반적인 추천 시스템





No negative feedback, inherently noisy, indicates confidence, appropriate measure

Collaborative Filtering Approach

Neighborhood Models

→ Item간 관계 또는 사용자들간 관계를 계산하는 방식
→ Item oriented approach : “ neighboring” item 점수기준
→ User oriented approach : Figure 1 참고.

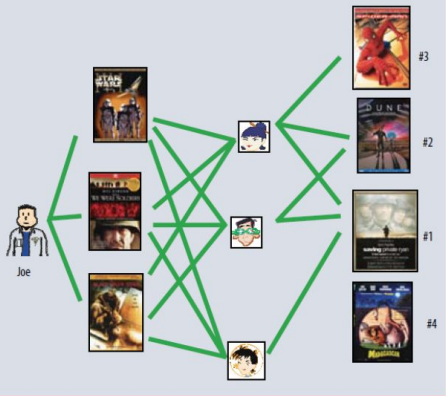


Figure 1. The user-oriented neighborhood method. Joe likes the three movies on the left. To make a prediction for him, the system finds similar users who also liked those movies, and then determines which other movies they liked. In this case, all three liked *Saving Private Ryan*, so that is the first recommendation. Two of them liked *Dune*, so that is next, and so on.

Figure1

Latent Factor Models

→ Rating 패턴을 factor를 추출하는 방식
→ 단, 해석하기 어려운 factor가 추출될 될 수도 있음
→ Figure 2 참고

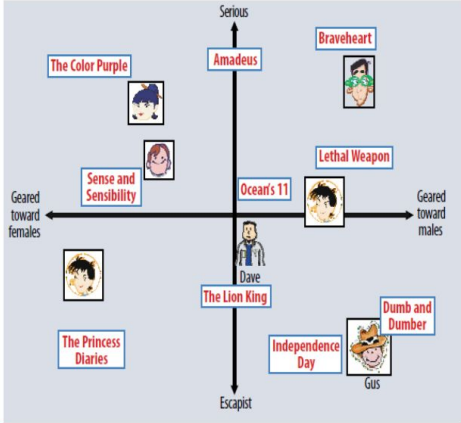


Figure 2. A simplified illustration of the latent factor approach, which characterizes both users and movies using two axes—male versus female and serious versus escapist.

Figure2

협업 필터링 방식은 기존 사용자의 행태 데이터가 쌓였을 경우에만 사용할 수 있다. 만약 아무도 채용 공고에 지원하지 않았다면 협업 필터링을 이용한 방법에서는 나에게 추천이 뜨지 않을 수 있다. 이러한 문제가 바로 콜드 스타트 문제이다.
새로운 아이템에 대해서는 유사도를 계산할 수 없기 때문에 추천이 작동하지 않는다.
이러한 경우 사용할 수 있는 방법이 Contents Based Filtering 이다.

Category

\+ Time

\+ User

\+ Items

Relations

Filter Weight

Items

Recommend

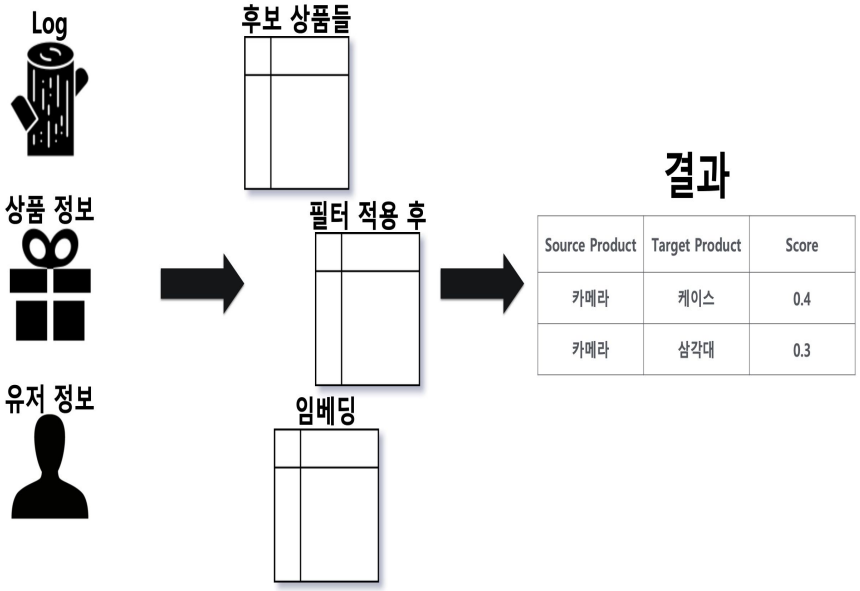
UI

	현재	미래
Input	상품	상품, 유저, 카테고리 등
Output	상품	상품, 추천 영역 등
Feature	위험도, 변동성	모델 점수들, 개인화 등

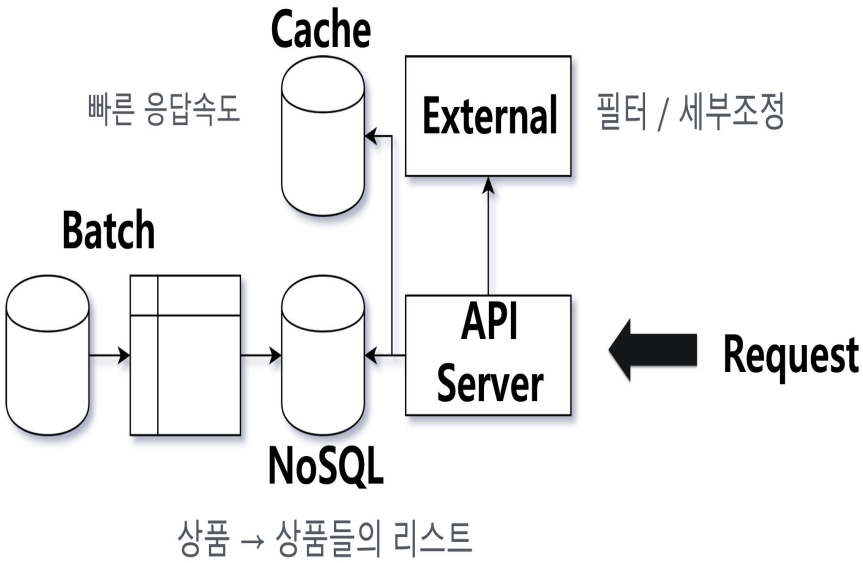
Item Relations Item

Relations = Model → Service

아키텍처 구성도



Source Product	Target Product	Score
카메라	케이스	0.4
카메라	삼각대	0.3



상품 정보 등은 데이터가 생성될 때만 가용

점진적인 개선이 힘들다.
모델의 재활용이 어렵다.

개선해야 할 점

1. 추천 모델과 서비스를 분리시킬 것
2. 상품정보나 유저정보를 서빙타임에 접근 가능할 것
3. 필터,부스팅등의 변경이 쉽고 빠를 것.

검색	자연어	키워드 매칭	관련도
추천	상품/유저	모델 결과	모델 스코어

쿼리 → 후보 찾기 → 랭킹

연관성 피처	상품 자체 피쳐
● 추천 모델 결과 ● 이미지 유사도 ● 카테고리 관련성 ...	● 카테고리 ● 상품평 ● 가격 ...

실시간 개인화

상품 연관성 피처	상품 피처	사용자 연관성 피처	사용자 피처
• 모델 점수 • 가격 차이는 얼마나? • 같은 카테고리인가? • 이미지 유사도는? …	• 상품평 개수 • 별점 • 트렌딩 모델 점수 • 실시간 피드백 …	• 세그먼트 별 피드백 • 선호 카테고리인가 …	• 기타 등등 …

↓

모델

↓

점수

THANK YOU